

實驗五

*Memory block –
substitute dual-port memory by
one port memory*

學生: 謝旻錡

學號: 313512078

日期: 2025/04/12

1. 實驗目的

學習使用 Quatus 內提供的 RAM 模組，並學習 RAM 的應用。

本此實驗 demo 目標是，使用兩個 one port ram 達成 two port 的功能。One port ram 的 address 同時控制讀寫，同一時間內寫入 a 資料且同時讀出 b 資料，而 two port ram 有能力可以同一時間內寫入 a 資料且同時讀出 b 資料，達到讀寫各有自己的 address port。

2. 實驗程式碼

Part5 Demo 題目，為使用 2 個 1-port RAM 組合成 2-port RAM。藉此了解 2-port 架構及其與 1-port 之差異。

Table 1.Input/Output

Input		Output	
CLOCK_50	CLK	LEDR0	寫入使能訊號指示燈
SW[4:0]	5bit 寫入位址訊號	HEX3-2	目前讀出位址(以 16 進制表示)
SW[8:5]	4bit 寫入數值訊號	HEX1-0	目前讀出數值(以 16 進制表示)
SW[9]	寫入使能訊號		

主要功能及架構:

我們想要一個能每秒更新顯示 RAM 中儲存的資料的顯示器，並且可以在不影響顯示的情況下輸入修改 RAM 中的儲存值。

因此我們會需要 a. 計數器、b. 2-port RAM、c. 七段顯示器，這三個電路來完成功能。

a. **時間計數器** (顯示規格: 讀出位址請每秒+1 更新，計數到 31 時歸零)

計數器 clk 為 50MHz，每 50M 為一秒鐘 (clk_count == sec, sec = 26'd49999999)，

```

//timer
reg [7:0] timer;
reg [25:0] clk_count;
parameter sec = 26'd49999999;
always @(posedge clk)begin
    if (timer == 8'd31 & clk_count == sec)begin
        timer <= 0;
        clk_count <= 0;
    end
    else if (clk_count == sec)begin //1sec
        clk_count <= 0;
        timer <= timer + 1;
    end
    else begin
        clk_count <= clk_count + 1;
    end
end
end

```

Figure 1. time counter

b. 2-port RAM

2-port RAM 主架構:

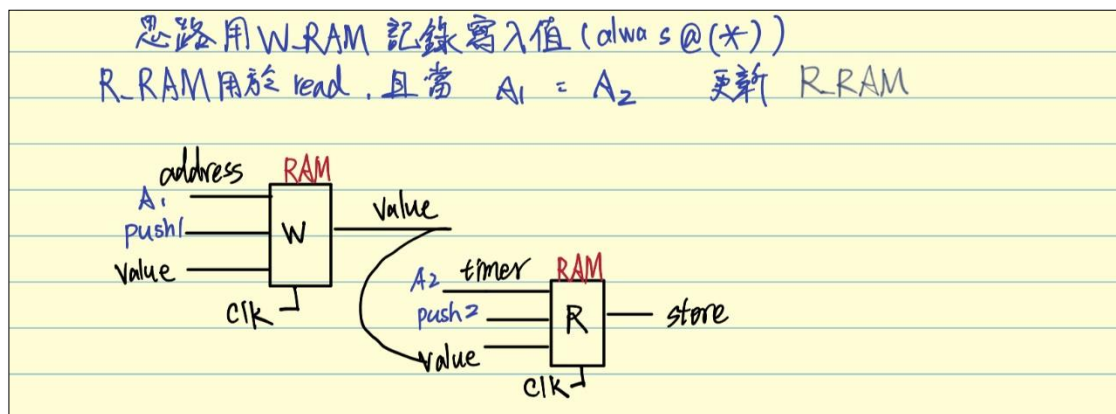


Figure 2. 2-port RAM 主架構

宣告兩個 1-port RAM，一個 RAM 用於寫另一個 RAM 用於讀。

```

//define
//r1 for write
ramlport2 r1(
    .address(a1),
    .clock(clk),
    .data(datain),
    .wren(push1),
    .q(q1)
);
//r2 for read
ramlport2 r2(
    .address(a2), //timer[4:0] for read
    .clock(clk),
    .data(q1),
    .wren(push2),
    .q(q2)
);

```

Figure 3. write RAM/ read RAM

由 Figure 2. 2-port RAM 主架構的思路主要參考 write back 的資料更新模式，write_RAM 負責記錄寫入的資料，但 read_RAM 不同時更新，直到要循環到要顯示此地址的資料時，才將 write_RAM 中的資料更新至 read_RAM 中。

Write back 資料更新模式為電腦中 cache 和 memory 資料交換的一種模式，其好處是當 cache 更新時，memory 不會同時更新，而是標記成 dirty cache，當 cache 要被寫回 memory 時才會更新資料至 memory 中，其好處是提高硬體間資料通訊效率。

我參考此方式，其資料間非同步更新的方式，來使我的 read_RAM 輸出，不會因 write in 新資料而被中斷干擾。

因此我需要兩個分開的寫入訊號(push1/push2)用來分別控制 write_RAM 和 read_RAM，如 Figure 4. 觸發時序圖和 Figure 5. 觸發訊號設計表示。Push1 為 write_RAM 的寫入訊號，Push2 為 read_RAM 的寫入訊號，兩者關係為 $push2 = \sim push1$ ，表示平時 read_RAM 會隨 time 持續更新 write_RAM 中的儲存值，但當 write_RAM 更新時($push1 == 1$)，read_RAM 就不會隨著更新($push2 == 0$)，不受 write_RAM 干擾。

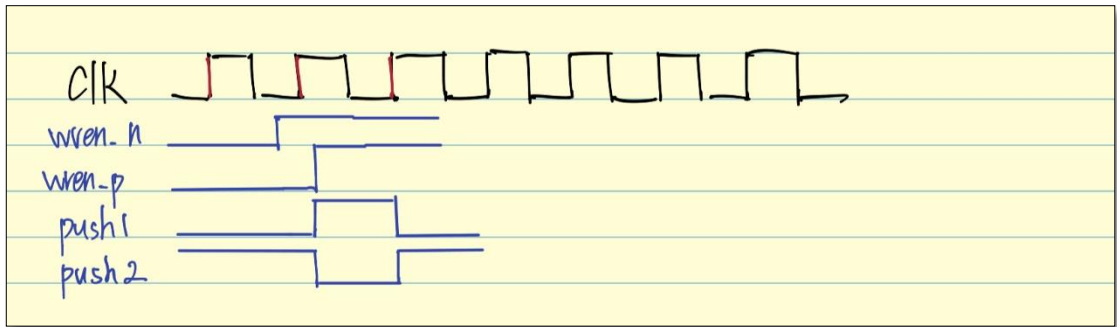


Figure 4. 觸發時序圖

```
// wren address switch
reg wren_now, wren_past;
always @(posedge clk)begin
    wren_now <= wren;
    wren_past <= wren_now;
end
reg push1, push2;
// push1/push2
always @(*)begin
    push1 = (wren_now & ~wren_past);
    push2 = ~push1;
end

// address
wire [4:0] a1,a2;
assign a1 = (push1)? address : timer[4:0]; //write address
assign a2 = timer[4:0]; //read address
```

Figure 5. 觸發訊號設計

c. 七段顯示器模組(16 進制)

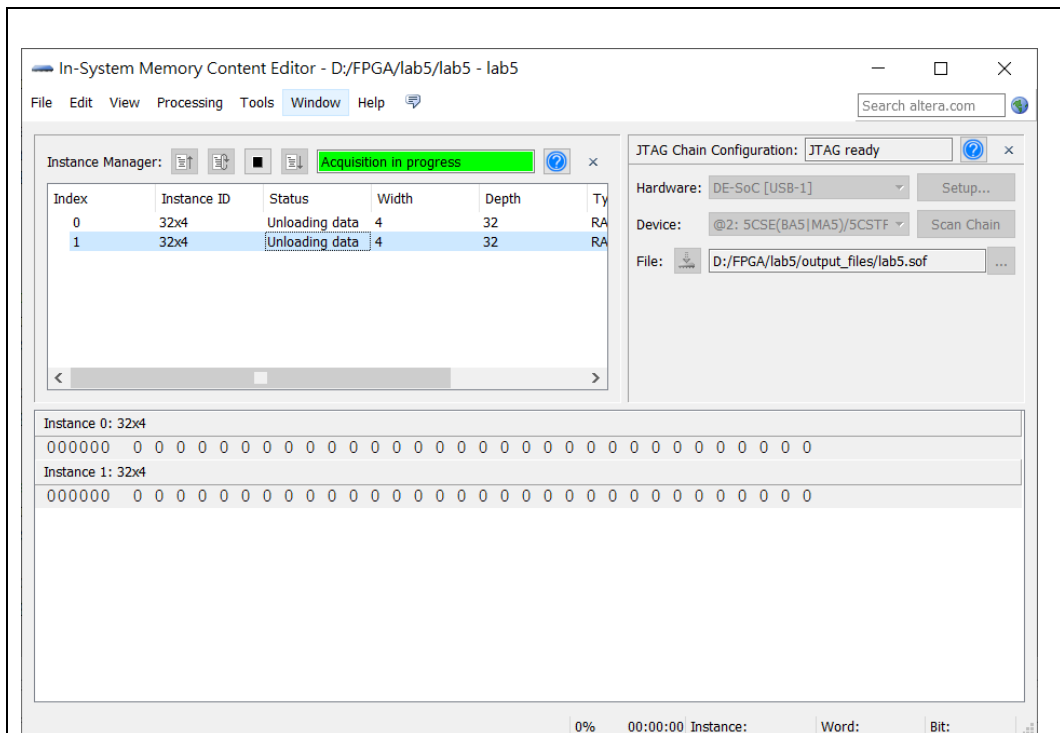
```
module HEX (
    input [3:0] hex,    // 4-bit 十六進制輸入
    output reg [6:0] seg // 7 段顯示器輸出
);
    always @(*) begin
        case (hex)
            4'h0: seg = 7'b1000000; // 0
            4'h1: seg = 7'b1111001; // 1
            4'h2: seg = 7'b0100100; // 2
            4'h3: seg = 7'b0110000; // 3
            4'h4: seg = 7'b0011001; // 4
            4'h5: seg = 7'b0010010; // 5
            4'h6: seg = 7'b0000010; // 6
            4'h7: seg = 7'b1111000; // 7
            4'h8: seg = 7'b0000000; // 8
            4'h9: seg = 7'b0010000; // 9
            4'hA: seg = 7'b0001000; // A
            4'hB: seg = 7'b0000011; // B
            4'hC: seg = 7'b1000110; // C
            4'hD: seg = 7'b0100001; // D
            4'hE: seg = 7'b0000110; // E
            4'hF: seg = 7'b0001110; // F
            default: seg = 7'b1000000; // 預設關閉
        endcase
    end
endmodule
```

Figure 6. HEX module

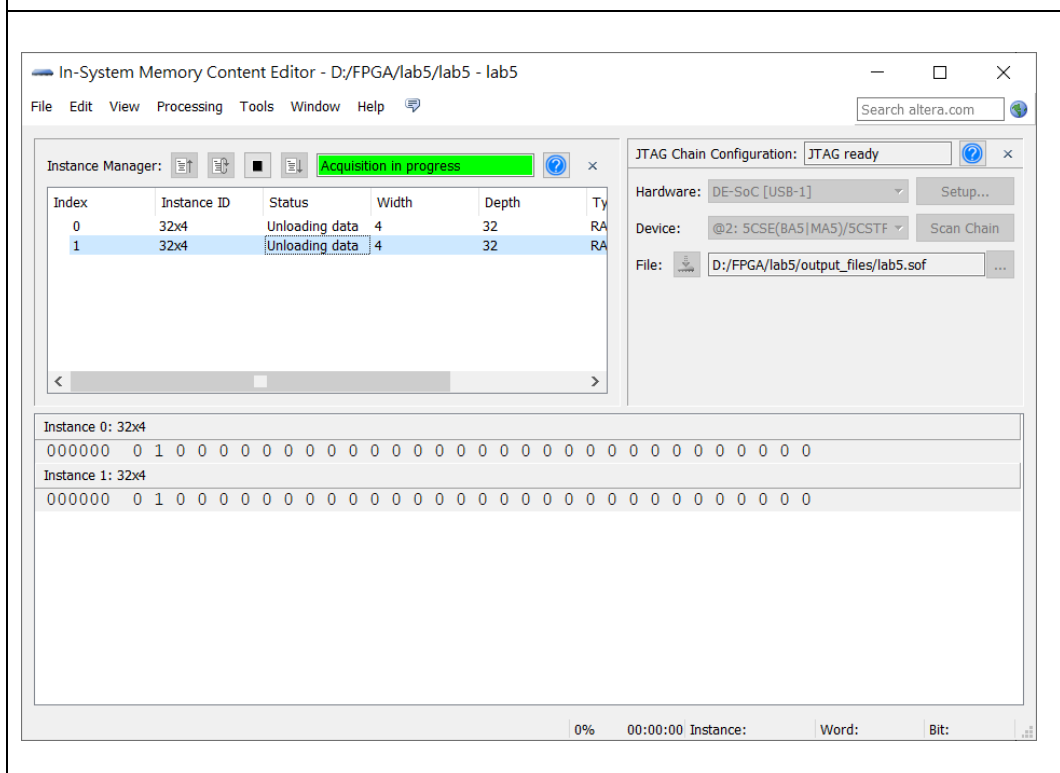
```
//HEX
HEX U3(timer[7:4], HEX3);
HEX U2(timer[3:0], HEX2);
HEX U1(4'h0, HEX1);
HEX U0(q2, HEX0);
```

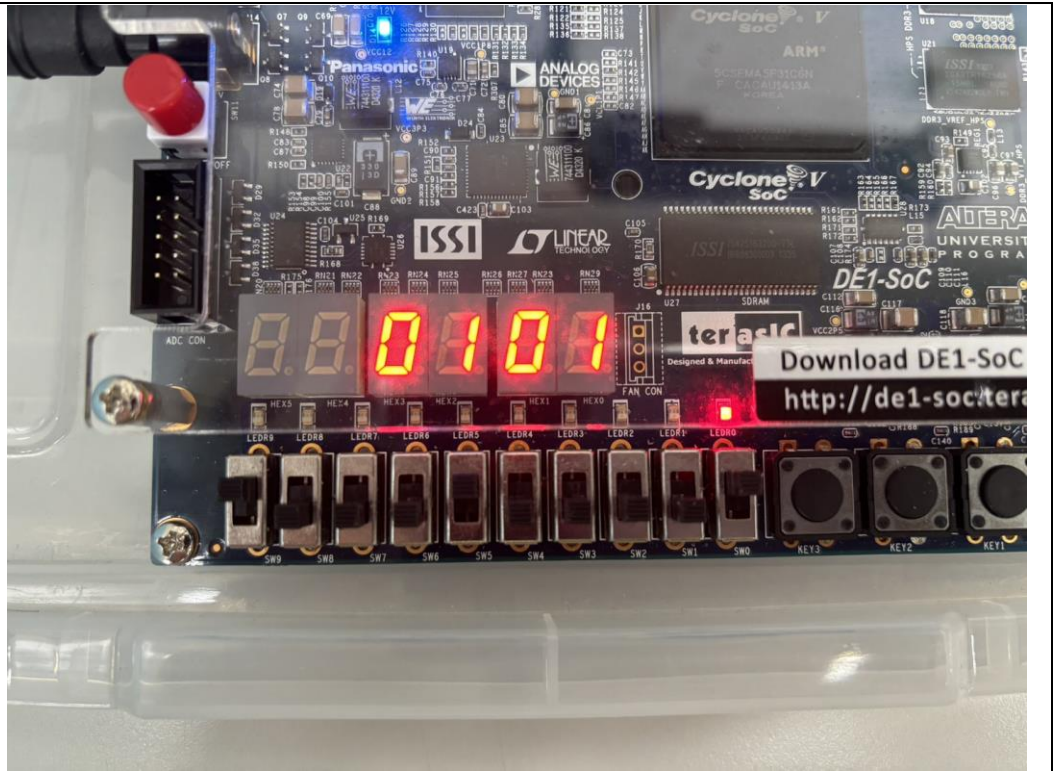
Figure 7. 七段顯示器輸出

3. 實驗結果照片(optional)

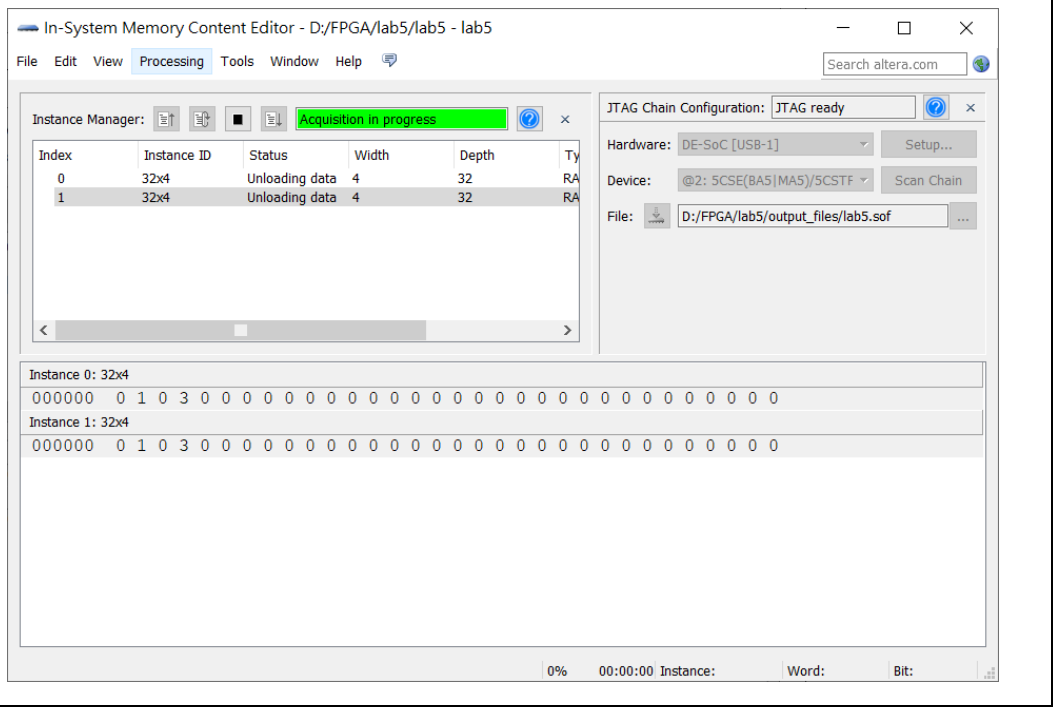


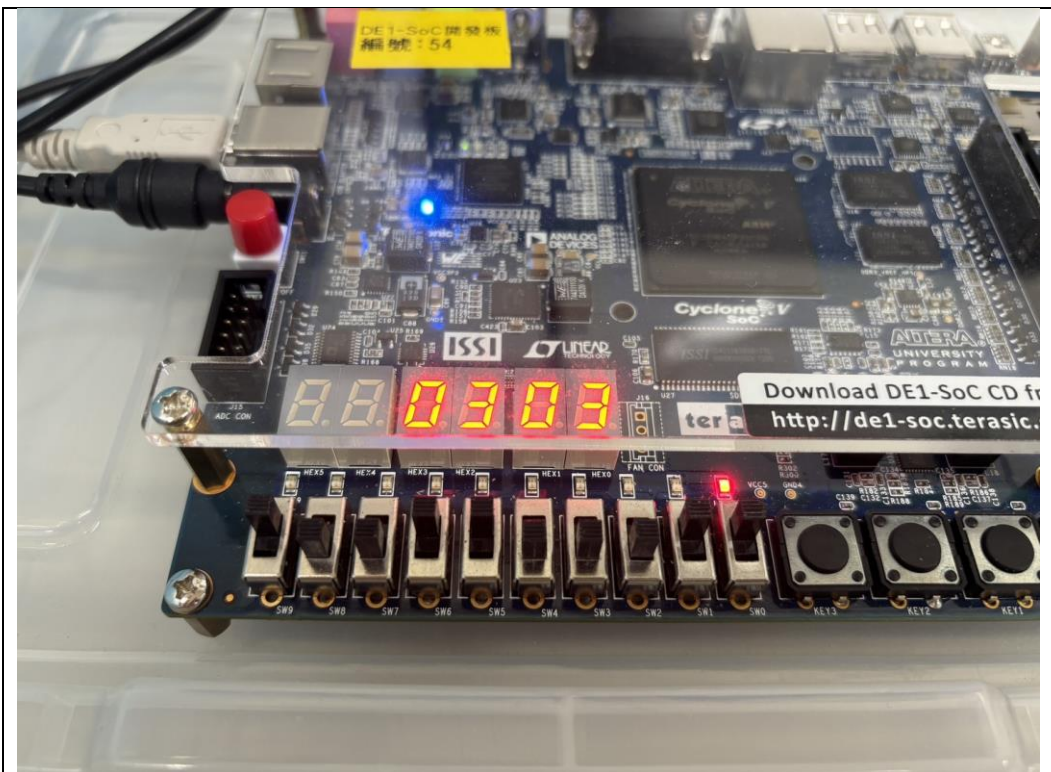
初始值全為零



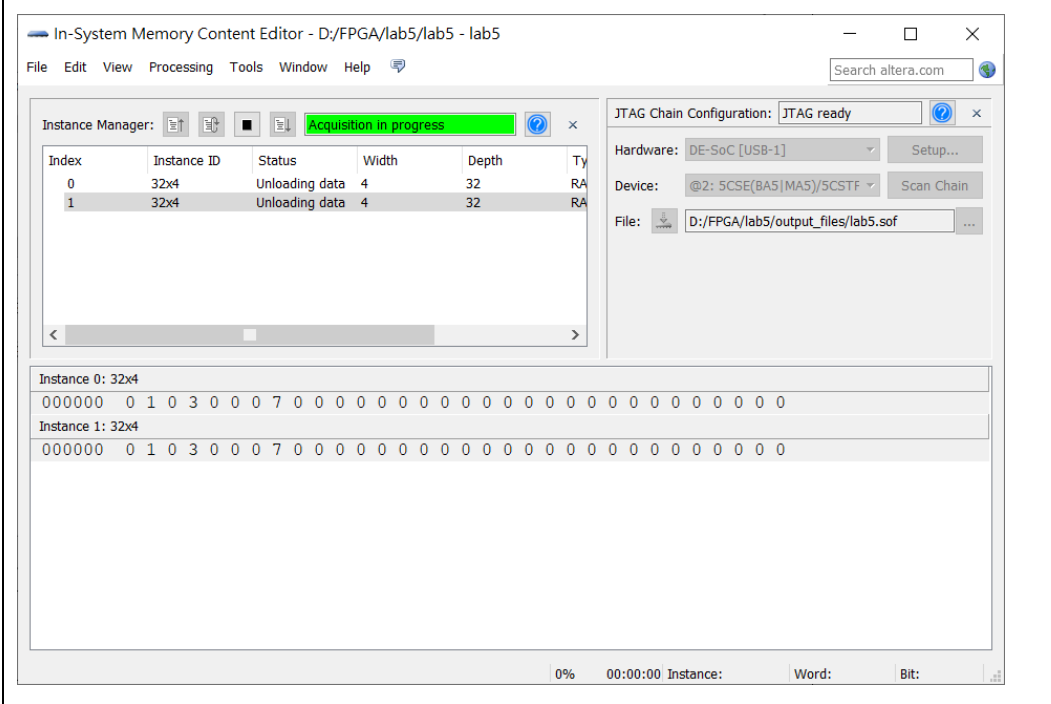


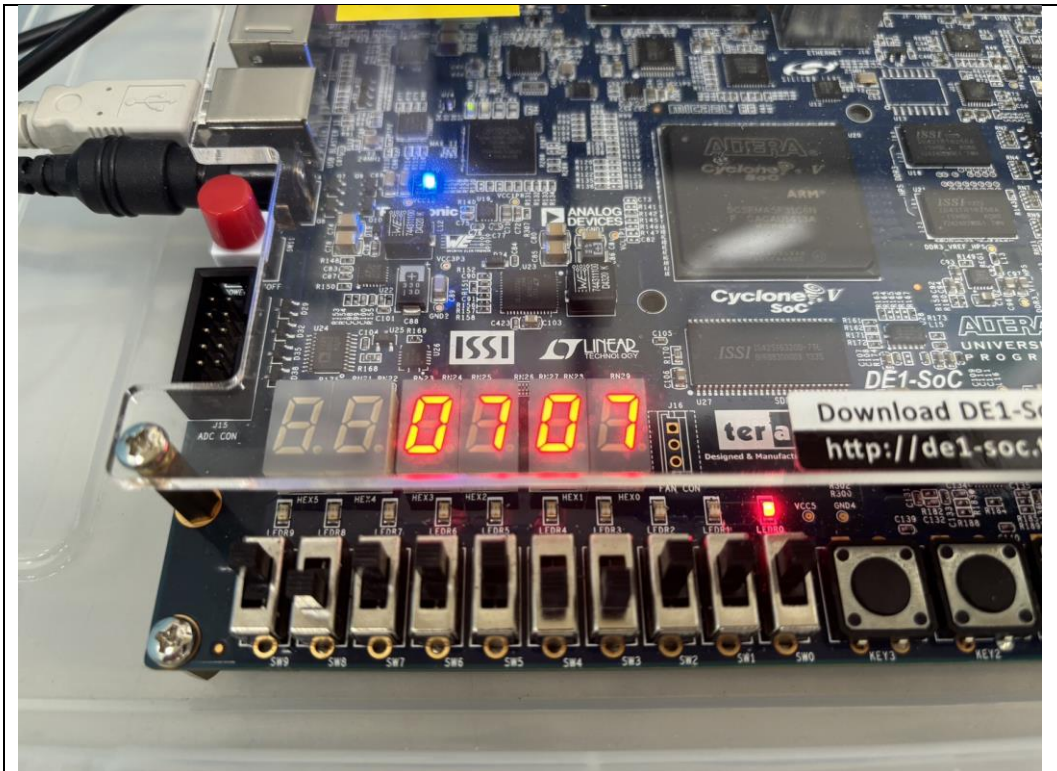
地址 1 中儲存 1



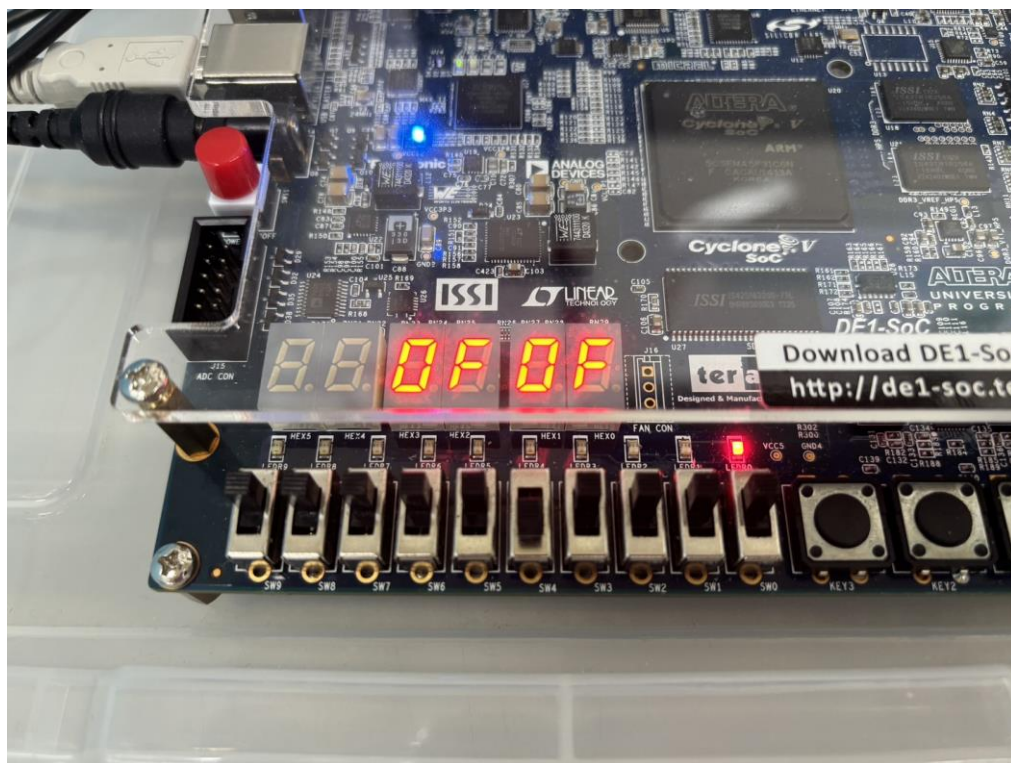
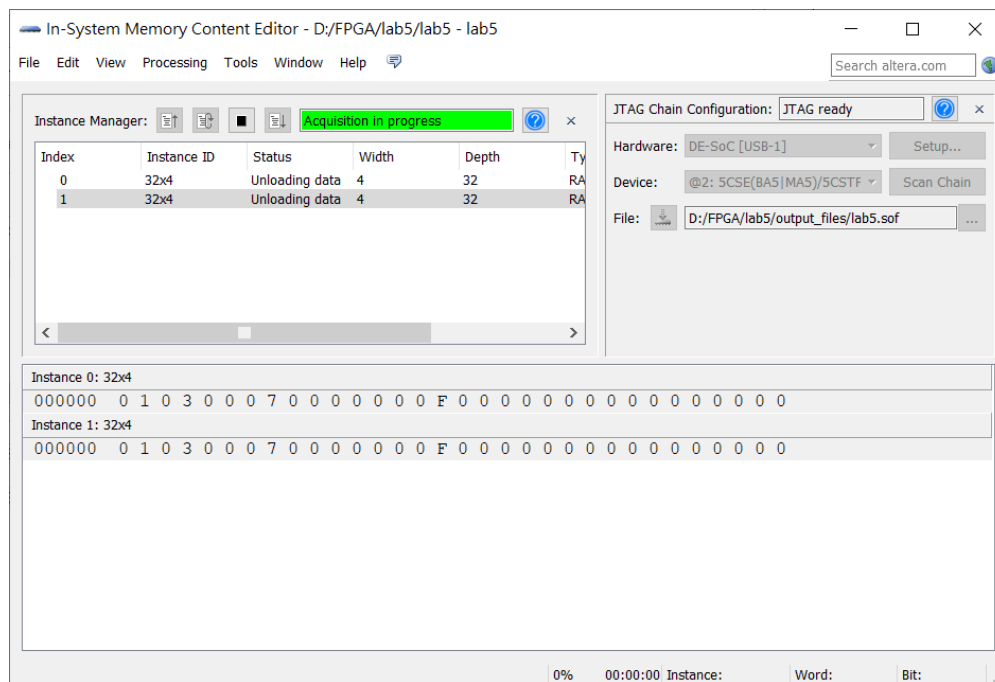


地址 3 中儲存 3

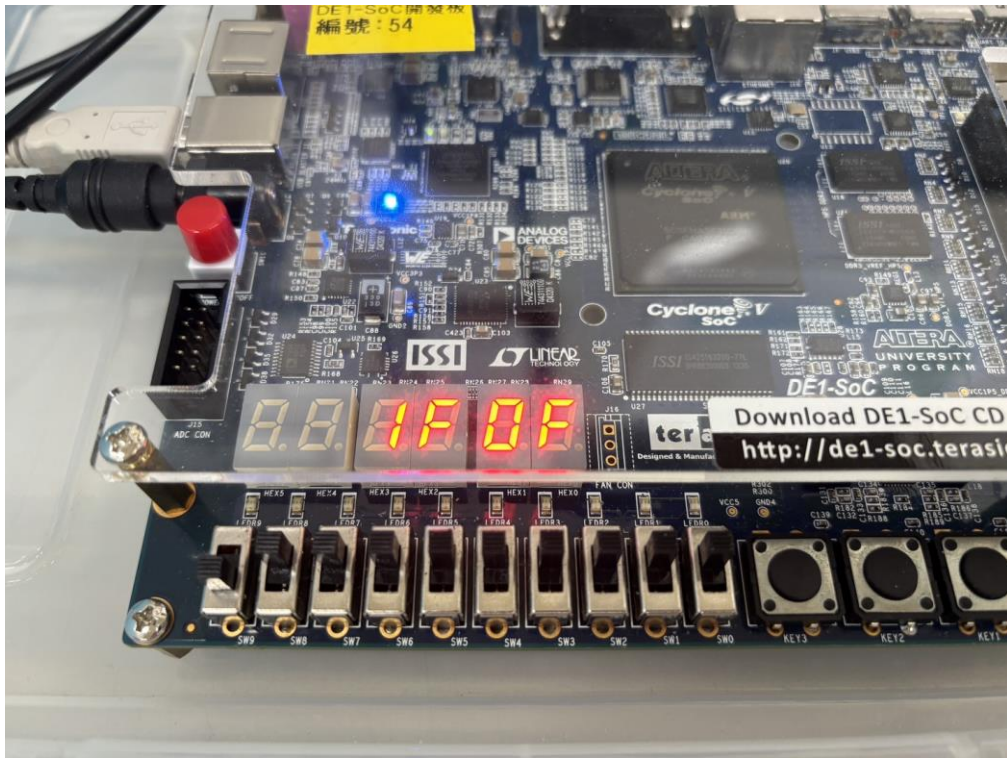
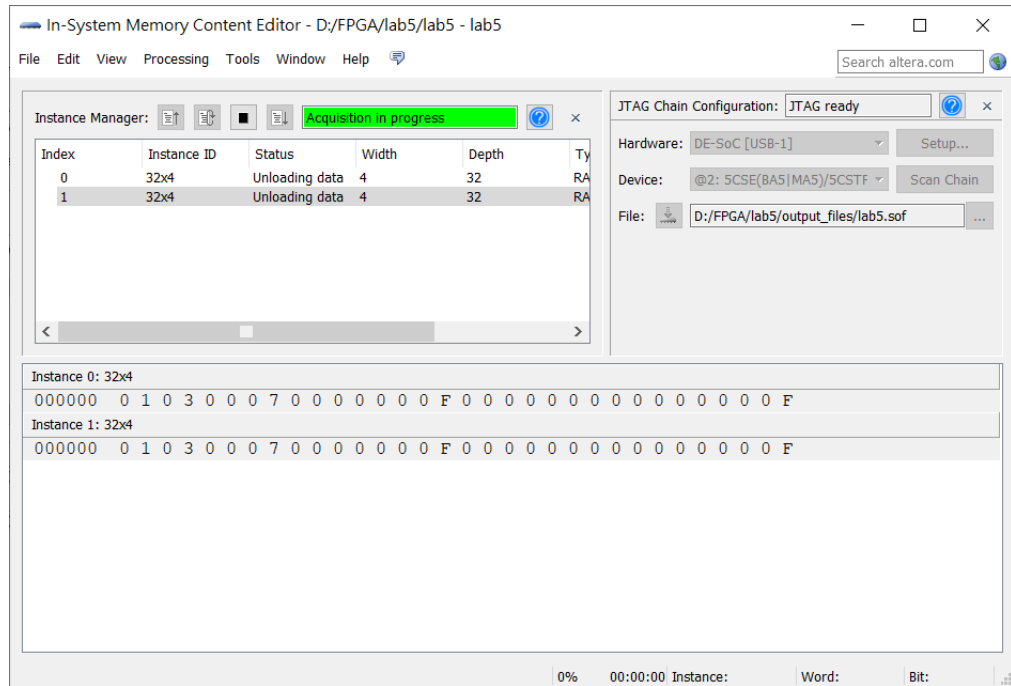




地址 7 中儲存 7

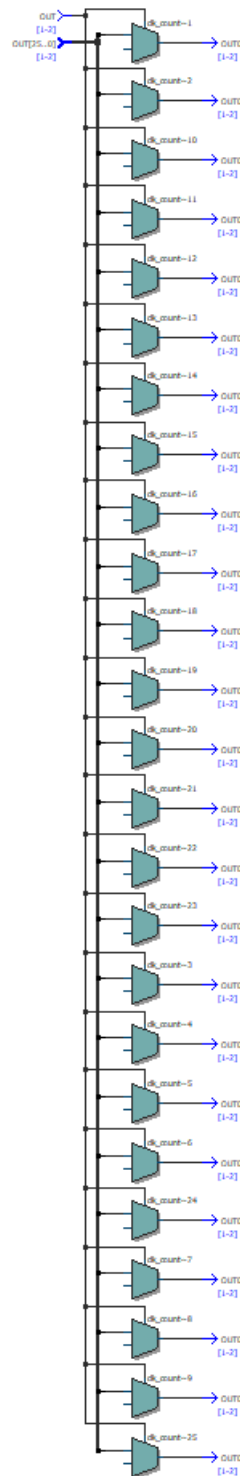


地址 15 中儲存 15

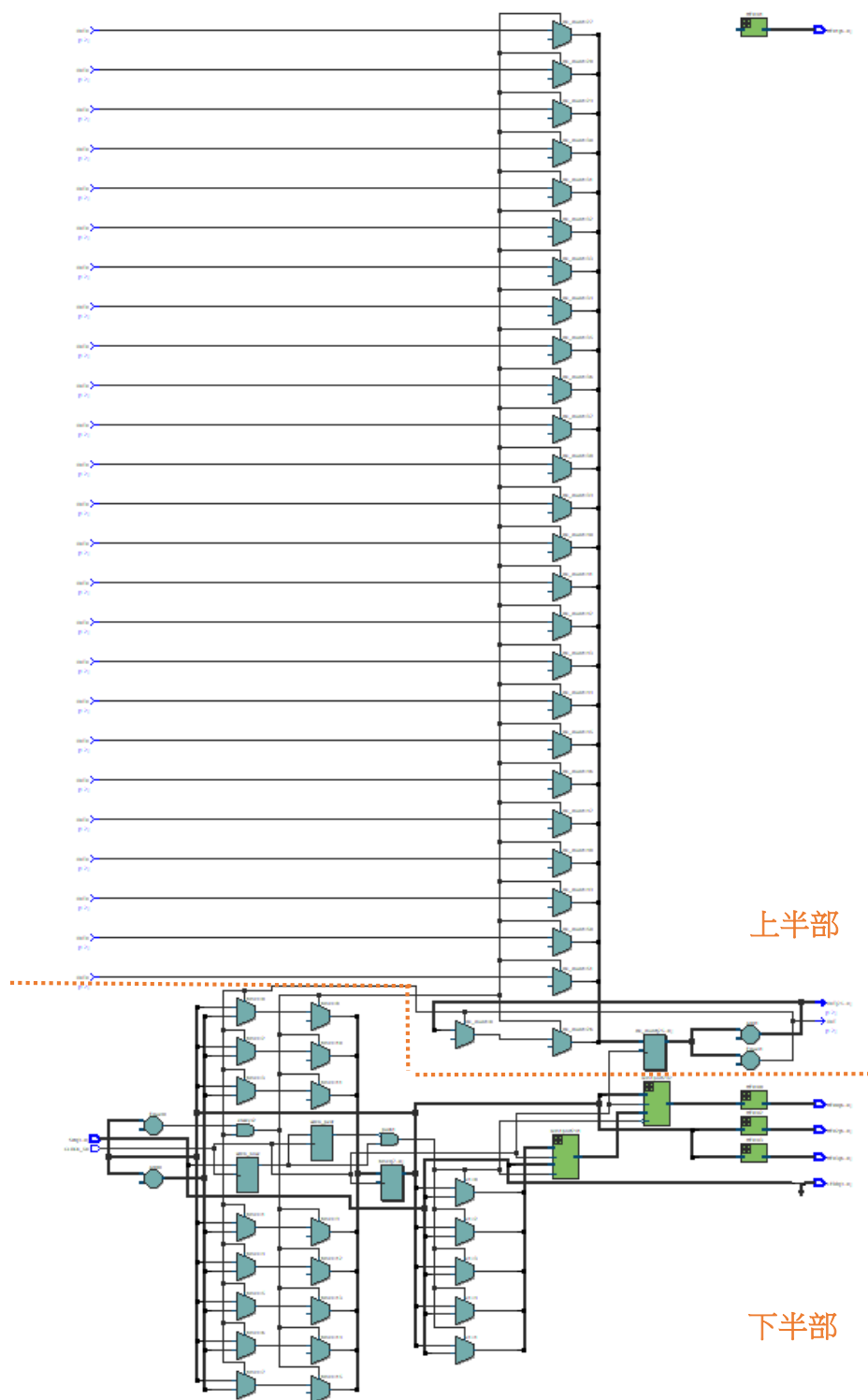


地址 31 中儲存 15

4. RTL 布局(optional)



計時器的計數器 RTL 前半電路



上半部: 計時器的計數器 RTL 後半電路

下半部: dual-port RAM 主電路

5. 問題與討論

如果要以此 demo 的模組進行更多的延伸應用，須注意 **writer** 及 **read** 中的資料是不同步的，**read** 中的資料是較慢更新的(最多時差一個循環 31 秒)，所以如果需要最新的即時資料用於計算或判斷時，那注意要取 **write** 中的值，而非 **read** 中的值。